Phase 0 프롬프트 (Claude Code용)

> **사용법:** 아래 `===` 블록 전체를 복사해서 Claude Code에 그대로 붙여넣으세요. 프로젝트 루트에서 Claude Code 실행 상태여야 합니다.

===
[역할]
너는 SajuCandle 프로젝트의 실행자(Executor)다. 설계자는 나(사용자)다.
architect-executor 워크플로우를 엄격히 따른다:
- 내가 "구현해"라고 명시적으로 지시하기 전까지 서비스 코드를 수정하지 않는다.
- 모든 산출물은 채팅에 장문으로 쓰지 말고 .md 파일로 작성한다.
- CLAUDE.md가 있으면 최우선으로 읽고 거기 정의된 용어·컨벤션을 따른다.

[배경]
SajuCandle은 심볼에 대해 사주 + 차트분석 기반 signal을 생성하는 웹 서비스다.
핵심 로직: 3TF OHLCV → analyze() → composite_score(0~100) → signal_grade(강진입/진입/관망/회피) → (조건부) TradeSetup
현재 main 브랜치는 "Week 10 Phase 2" 상태로 알려져 있으나, 설계 문서(명세)와 실제 코드의 일치 여부가 확인되지 않았다.

지금 우리는 로직 전반 품질을 67점(100점 만점)에서 85점 이상으로 끌어올리는 대규모 개선 작업을 막 시작했다.
이 작업은 Phase 0 ~ Phase 4로 나뉜다:
- Phase 0: 현황 파악 및 CLAUDE.md 갱신 (지금 이 작업)
- Phase 1: 백테스트 하네스 구축
- Phase 2: 숏 신호 대칭 구현 (5등급 체계 도입)
- Phase 3: 지표 고도화 (RSI divergence, Volatility regime, BREAKOUT 재검증)
- Phase 4: composite 가중치/임계값 튜닝

Phase 0의 목적은 "Phase 1~4가 기댈 수 있는 정확한 출발점"을 확보하는 것이다.
명세 문서에 근거한 가정이 실제 코드와 어긋나면 이후 모든 Phase가 뒤틀린다.

[Phase 0 목표]
1. 설계 명세와 실제 main 브랜치 코드 간 괴리를 빠짐없이 찾아낸다.
2. CLAUDE.md가 현재 코드 기준으로 최신이 되도록 갱신한다.
3. 이후 Phase들이 참조할 "코드 기준 사실(truth)"을 문서로 확정한다.

[산출물]
다음 두 파일을 작성한다. 프로젝트 루트 기준 경로:
- `docs/planning/research/phase0_current_state.md` (명세↔코드 대조 리서치)
- `CLAUDE.md` (갱신판. 기존 파일이 있으면 덮어쓰되, 기존 내용에서 여전히 유효한 항목은 보존)

위 디렉토리가 없으면 생성한다.

[작업 순서]
architect-executor 규칙을 따라 아래 순서로 진행한다.

Step 1: 프로젝트 컨텍스트 동기화
- 프로젝트 루트의 기존 CLAUDE.md를 읽는다 (있다면).
- 기존 내용 중 보존할 항목과 갱신할 항목을 판단할 수 있도록 요약만 기억한다 (채팅에 쓰지 않음).

```
- README, pyproject.toml, requirements.txt, 디렉토리 구조를 훑어 프로젝트 개요를 파악한다.

## Step 2: 코드 리서치
다음 파일들을 읽고 명세 대비 실제 상태를 확인한다.

### 필수 읽기 대상
```

src/sajucandle/analysis/swing.py
src/sajucandle/analysis/structure.py
src/sajucandle/analysis/timeframe.py
src/sajucandle/analysis/multi_timeframe.py
src/sajucandle/analysis/composite.py
src/sajucandle/analysis/support_resistance.py
src/sajucandle/analysis/volume_profile.py
src/sajucandle/analysis/trade_setup.py
src/sajucandle/signal_service.py
src/sajucandle/tech_analysis.py
src/sajucandle/handlers.py
src/sajucandle/market/router.py

경로가 명세와 다르면 실제 경로를 찾아 읽고, 차이는 리포트에 기록한다.

각 파일에서 확인할 사실
아래는 Phase 0 리서치에서 **반드시 확인해야 하는 체크리스트**다. 각 항목에 대해 "명세값 / 코드값 / 일치 여부"를 기록한다.

A. Swing 감지 (swing.py)
- [ ] `fractal_window` 상수값 (명세: 5)
- [ ] `atr_multiplier` 기본값 (명세: 1.5)
- [ ] `atr_period` (명세: 14)
- [ ] ATR 계산이 Wilder smoothing인지 (첫 14봉 SMA → 이후 (avg*13 + TR)/14)
- [ ] Prominence 비교 기준이 좌/우 각 10봉 high/low인지
- [ ] 봉 수 < 11일 때 빈 리스트 반환하는지
- [ ] `atr_multiplier=0`일 때 필터 off 처리 있는지

B. Structure 판정 (structure.py)
- [ ] 판정 우선순위: UPTREND → BREAKDOWN → BREAKOUT → DOWNTREND → RANGE 순서인지
- [ ] 각 state의 점수 매핑 (UPTREND=70, BREAKOUT=80, RANGE=50, BREAKDOWN=30, DOWNTREND=20)
- [ ] UPTREND 조건: highs≥3 AND lows≥3 AND 최근 3개 HH + 3개 HL
- [ ] BREAKOUT 조건: highs≥3 AND 최근 고점 > max(이전 고점들) × 1.03
- [ ] BREAKDOWN 조건: highs≥2 AND lows≥3 AND 최근 high 상승 AND 최근 low 하락
- [ ] 반환 타입: `StructureAnalysis(state, last_high, last_low, score)`

C. Timeframe 트렌드 (timeframe.py)
- [ ] EMA50 계산 방식 (초기 SMA50, 이후 α=2/51)
- [ ] threshold = last_ema × 0.0001 (0.01%)
- [ ] UP 조건: above AND rising AND close_rising
- [ ] DOWN 조건: below AND falling AND close_falling
- [ ] 봉 수 < 56일 때 FLAT 반환
- [ ] EMA 기울기 판정을 5봉 전(`ema[-6]`) 기준으로 하는지

D. Alignment (multi_timeframe.py)
- [ ] ups/downs 카운트 방식
- [ ] score 공식: `round((ups - downs + 3) / 6 × 100)`
- [ ] aligned + bullish일 때 `max(score, 90)` 보정
- [ ] aligned + bearish일 때 `min(score, 10)` 보정

E. 보조 지표 (composite.py 또는 tech_analysis.py)
- [ ] RSI(14) Wilder 표준 구현
- [ ] RSI → score 매핑 구간 (≤30→70, ≤45→55, ≤55→50, ≤70→40, >70→20)
- [ ] volume_ratio = `volumes[-1] / mean(volumes[-21:-1])`
- [ ] vol ratio → score 매핑 (≥1.5→65, ≥1.0→55, ≥0.5→45, <0.5→35)

F. Composite 조합 (composite.py)

- [ ] 1d swing 없을 때 1h 폴백 로직
- [ ] swing=[]일 때 structure score에 alignment 50% 섞는 보정
- [ ] 최종 가중합: 0.45 × structure + 0.35 × alignment + 0.10 × rsi + 0.10 × volume
- [ ] clamp(0, 100)

G. 사주 + 차트 최종 합산 (signal_service.py)
- [ ] 사주 composite 가중치 0.1
- [ ] 차트 analysis composite 가중치 0.9
- [ ] final = round(0.1 × saju + 0.9 × analysis), clamp(0, 100)

H. 등급 판정 (_grade_signal, signal_service.py 또는 composite.py)
- [ ] 강진입 조건: score≥75 AND aligned AND state in (UPTREND, BREAKOUT)
- [ ] Week 10 Phase 2 게이팅: state in (DOWNTREND, BREAKDOWN) AND score≥60 → "관망"
- [ ] 임계값: ≥60 진입, ≥40 관망, <40 회피

I. Trade Setup (trade_setup.py)
- [ ] `_SL_ATR_MULT = 1.5`, `_TP1_ATR_MULT = 1.5`, `_TP2_ATR_MULT = 3.0`
- [ ] `_SNAP_TOLERANCE = 0.3`, `_SNAP_TOLERANCE_TP2 = 0.5`
- [ ] `_SR_BUFFER_ATR = 0.2`
- [ ] SL search range: `[entry - 1.8×ATR, entry - 1.2×ATR]`
- [ ] TP2 search range: `[entry + 2.5×ATR, entry + 3.5×ATR]`
- [ ] ATR ≤ 0일 때 `atr = entry × 0.01` 폴백
- [ ] 반환 필드: entry, stop_loss, take_profit_1, take_profit_2, risk_pct, rr_tp1, rr_tp2, sl_basis, tp1_basis, tp2_basis

J. S/R 식별 (support_resistance.py + volume_profile.py)
- [ ] Volume profile bucket_count = 20
- [ ] 상위 5개 노드 선택
- [ ] 최대 volume node strength="medium", 나머지 "low"
- [ ] 클러스터 병합 거리 ≤ 0.5%
- [ ] swing + volume 둘 다 소스면 strength="high"
- [ ] 현재가 위/아래 각 최대 3개 반환

K. 등급별 카드 출력 (handlers.py)
- [ ] 진입/강진입 카드에 "세팅" 블록 포함 (entry/SL/TP1/TP2/R:R/risk%)
- [ ] 관망/회피 카드에 "주요 레벨" 블록 포함 (저항·지지 각 3개)
- [ ] 한국어 라벨 포맷 확인

L. 지원 심볼 (market/router.py)
- [ ] `_CRYPTO_SYMBOLS`: BTCUSDT, ETHUSDT, XRPUSDT
- [ ] `_STOCK_SYMBOLS`: AAPL, MSFT, GOOGL, NVDA, TSLA, AMD, META, AMZN
- [ ] 미지원 심볼에 `UnsupportedTicker` 예외

M. 캐시 (tech_analysis.py 또는 별도 모듈)
- [ ] OHLCV 캐시 키 포맷: `ohlcv:{symbol}:{interval}:{fresh|backup}`
- [ ] Fresh TTL 1h, Backup TTL 24h
- [ ] Signal 캐시: `signal:{chat_id}:{date}:{ticker}`, TTL 300초

- [ ] 사주 캐시: `score:{chat_id}:{date}:{asset}`, TTL KST 자정까지

N. 테스트 인프라
- [ ] `tests/` 디렉토리 존재 여부
- [ ] 각 analysis 모듈별 unit test 존재 여부
- [ ] 테스트 러너 설정 (pytest 등)
- [ ] CI 설정 존재 여부

Step 3: 리서치 문서 작성
`docs/planning/research/phase0_current_state.md`를 다음 구조로 작성한다.

```markdown
# Phase 0 리서치: 현재 코드 상태 점검

작성 시점: YYYY-MM-DD
대상 브랜치: main (commit: <hash>)
작성 목적: 설계 명세(Week 10 Phase 2)와 실제 코드의 일치 여부 확인,
         Phase 1~4 작업의 기준점 확립

## 요약
- 전체 체크 항목 수: N
- 명세와 일치: X개
- 명세와 불일치: Y개 (상세는 §괴리 리스트)
- 명세에 없는 추가 로직: Z개

## 프로젝트 구조
(디렉토리 트리, 핵심 모듈 설명 2~3줄씩)

## 체크리스트 결과
### A. Swing 감지
| 항목 | 명세값 | 코드값 | 일치 | 위치 |
|---|---|---|---|---|
| fractal_window | 5 | 5 | ✓ | swing.py:12 |
| atr_multiplier | 1.5 | 1.5 | ✓ | swing.py:13 |
| ... | ... | ... | ... | ... |

(A~N 전체 체크리스트 결과. 불일치는 반드시 "코드값" 칸에 실제 값, "위치" 칸에 파일:라인)

## 괴리 리스트 (Critical)
명세와 달라서 이후 Phase에 영향을 줄 수 있는 항목만 정리.
각 항목:
- 무엇이 다른가
- 어느 Phase에 영향을 주는가
- 제안 조치 (명세를 따라 코드를 고칠지, 코드를 인정하고 명세를 갱신할지)

## 명세에 없는 추가 로직
명세에 언급되지 않았으나 코드에 존재하는 로직. 있으면 리스트업.
```

```
## 테스트 인프라 상태
- 테스트 파일 목록
- 커버리지 추정
- Phase 1 백테스트 하네스를 넣을 적절한 위치 제안

## 캐시·인프라 확인
- Redis 사용 여부 및 연결 방식
- OHLCV 프로바이더 구현 상태 (Binance, yfinance)

## 이후 Phase를 위한 권고사항
- Phase 1에서 주의할 점 (예: 룩어헤드 방지를 위해 고려할 캐시 레이어)
- Phase 2에서 주의할 점 (예: 숏 구현 시 영향 받을 부분 미리보기)

## Open Questions
리서치 중 명확하지 않아 설계자 판단이 필요한 항목. 있으면 질문 형태로.
```

```
## Step 4: CLAUDE.md 갱신 계획
리서치 결과를 바탕으로 CLAUDE.md 갱신안을 같은 `phase0_current_state.md` 문서 마지막에
"§CLAUDE.md 갱신 제안" 섹션으로 넣는다. 실제 CLAUDE.md 파일 수정은 설계자 승인 후에만 진행
한다.

갱신 제안 섹션 구조:
```markdown
## CLAUDE.md 갱신 제안

### 기존에서 보존할 내용
(불릿 리스트)

### 갱신이 필요한 내용
(현재 → 수정 제안)

### 새로 추가할 내용
- 도메인 용어 (예: swing, structure, alignment, composite_score, TradeSetup 등 정의)
- 코딩 컨벤션 (파일에서 관찰된 실제 컨벤션 기준)
- 모듈 책임 명세 (각 analysis/ 서브모듈이 무엇을 담당하는지)
- 제약사항 (예: 1h yfinance는 60일 제한, 미국장 공휴일 미처리 등)
- Phase별 작업 규칙 (research → plan → 승인 → 구현)
```

Step 5: 보고 및 대기
위 작업이 끝나면 다음 형식으로 보고하고 대기한다:
```

Phase 0 리서치 완료.

산출물:

- docs/planning/research/phase0_current_state.md

주요 발견사항 (3~5줄):

- ...

명세 대비 괴리 개수: X개 (critical Y개)

CLAUDE.md 갱신 제안을 리서치 문서 마지막 섹션에 포함시켰습니다.
검토 후 다음 중 하나를 지시해주세요:

1. CLAUDE.md 실제 파일을 갱신 제안대로 반영 (설계자 승인 시 수정)
2. 괴리 리스트에 대한 추가 조사
3. 바로 Phase 1 프롬프트 작성으로 진행

```
[금지 사항]
- 서비스 코드 수정 금지 (리서치만, 파일 생성은 리서치 산출물 .md만)
- 채팅에 리서치 결과를 장문으로 쓰지 말 것. .md 파일로 쓰고 채팅에는 경로와 요약만.
- 명세값과 코드값이 다를 때 임의로 판단하지 말 것. "사실 기록"만 하고 판단은 설계자에게.
- 새 기능 제안 금지. 이번 Phase는 "현황 파악"이 전부다.

[시작]
위 작업을 시작한다.
===
```

이 프롬프트 사용 시 참고

실행 방식

1. 프로젝트 루트에서 Claude Code 세션 시작
2. 위 `===` 블록 통째로 붙여넣기
3. Claude Code가 파일을 읽고 `docs/planning/research/phase0_current_state.md` 생성할 때까지 대기
4. 리서치 결과 검토
5. 괴리 리스트 보고 판단:
   - 명세가 맞다면 → Phase 1 전에 코드를 명세에 맞추는 별도 작업 지시
   - 코드가 맞다면 → 이 리서치 문서 자체가 Phase 1~4의 새 기준점이 됨
6. CLAUDE.md 갱신 제안 검토 후 반영 지시

주의

- **Phase 0에서 코드 수정을 지시하지 않는다.** 괴리가 발견돼도 이번 Phase 범위는 "확인과 기록"까지. 수정은 별도 판단 후 별도 지시.
- Claude Code가 모든 체크 항목을 한 번에 보고하지 못하고 중간에 멈추면, "계속 진행해"로 이어가거나 특정 섹션만 재확인 지시 가능.
- 리서치 문서가 너무 길어지면 A~N 섹션을 별도 파일로 분리 요청해도 됨.

다음 단계 (Phase 0 완료 후)

Phase 0 리서치 결과에서 나온 **실제 코드 기준 사실**을 반영해서 Phase 1 프롬프트를 작성한다. 마스터 플랜의 Phase 1 섹션을 기반으로 하되, Phase 0에서 발견된 주의사항(예: 실제 캐시 구조, 실제 provider 구현)을 프롬프트에 포함시킨다.